

Small Models, Strong Reasoning:

An Adaptive Hybrid Framework for Compute-Efficient Olympiad Mathematics

Minh Hùng Lê

September 2026

Abstract

Large language models have shown strong mathematical reasoning capability, but solving difficult Olympiad-style problems with a fixed end-to-end language-model pipeline remains expensive and unreliable. Long reasoning chains may hallucinate intermediate claims, malformed mathematical input can redirect the entire solution, and a second language model acting as a critic does not by itself provide a mathematical certificate. This work presents an adaptive hybrid reasoning framework for advanced mathematical problem solving that combines compact open-weight language models with mathematical input recovery, fine-grained problem-family routing, retrieval-augmented reasoning, deterministic exact solvers, symbolic computation, proof obligations, and fail-closed verification.

The proposed system uses a lightweight proposer, a 4B instruction-tuned solver, and a 4B thinking critic, while delegating supported mathematical structures to specialized exact engines. These engines cover relational algebra, extremal partitioning, Euclidean geometry, functional equations, combinatorial valuation, floor-sum valuation, geometry-sequence asymptotics, adaptive digit-sum dynamics, finite sequence correlation, and divisor-minimization asymptotics. The architecture is designed to allocate expensive neural computation only when exact reasoning is unavailable.

On the ten AIMO3 reference problems used during system development, the final system verifies all ten answers. Problem 1 is solved through semantic compilation and exact symbolic reasoning with one model call and 744 total tokens, while Problems 2–10 are solved through deterministic family engines with zero model calls. The recorded batch contains no verification false positives. These results demonstrate the engineering feasibility of combining small language models with structured mathematical tools and adaptive computation. However, because the reference set was used iteratively during development, the reported result should be interpreted as engineering validation rather than an unbiased estimate of generalization. The next scientific step is to freeze the architecture and evaluate it on unseen Olympiad problems against fixed-compute baselines.

Keywords: mathematical reasoning; Olympiad problems; small language models; adaptive computation; deterministic solvers; retrieval-augmented generation; symbolic reasoning; verification

1 Introduction

Recent language models are increasingly capable of solving mathematical reasoning tasks, yet difficult Olympiad-style problems continue to expose several weaknesses of conventional end-to-end inference. A model may understand the broad topic of a problem while still misreading a mathematical expression, selecting the wrong theorem, committing to an unsupported candidate, or generating

a long explanation that appears plausible but lacks a decisive certificate. These failure modes become more severe when the input originates from PDF documents containing nested fractions, summations, superscripts, piecewise definitions, or geometric notation.

A second challenge is computational efficiency. Standard reasoning pipelines often allocate similar token budgets to easy and hard problems, even when some problems can be solved exactly with symbolic algebra, enumeration, dynamic programming, modular arithmetic, or a specialized deterministic algorithm. This produces an unfavorable trade-off: neural inference is used even when a cheaper and more reliable exact procedure is available.

This work investigates a different design philosophy. Instead of treating mathematical reasoning as a single language-model generation problem, we treat it as a *system-design problem*. The system first repairs and validates the mathematical input, identifies the structure of the problem, and then selects the least expensive reliable reasoning mechanism. If the problem belongs to a supported exact family, a deterministic engine attempts to solve it and produce a certificate. Otherwise, the system falls back to retrieval-augmented language-model reasoning, focused proof obligations, repair, and critic-based review.

The central research hypothesis is

$$\text{Accuracy}_{\text{adaptive}} \geq \text{Accuracy}_{\text{fixed}}, \quad (1)$$

while ideally satisfying

$$\text{Compute}_{\text{adaptive}} < \text{Compute}_{\text{fixed}}. \quad (2)$$

A stronger long-term objective is to determine whether a carefully structured 4B system can outperform a larger fixed-reasoning model while consuming fewer tokens, fewer model calls, and less GPU time.

The current implementation is designed for offline execution on two NVIDIA Tesla T4 GPUs. It uses a lightweight DeepSeek-R1-Distill-Qwen-1.5B proposer, Qwen3-4B-Instruct-2507 as the primary solver and reviser, and Qwen3-4B-Thinking-2507 as the high-cost reasoning critic. Deterministic reasoning runs on the CPU whenever possible.

The main contributions of this work are as follows:

1. We propose an adaptive hybrid architecture that combines language models with mathematical input recovery, fine-grained routing, retrieval, exact computation, proof obligations, and verification.
2. We introduce a math-fidelity layer designed to detect and repair malformed mathematical expressions before reasoning begins.
3. We develop a collection of fail-closed deterministic family engines covering a diverse set of Olympiad structures, including geometry, functional equations, combinatorial valuation, floor sums, sequence dynamics, polynomial correlation, and divisor minimization.
4. We separate candidate generation from verification and require exact certificates whenever a deterministic solver is used.
5. We show, on the AIMO3 reference set used during engineering development, that the final system verifies all ten problems while requiring only one neural call across the entire batch.
6. We identify the main scientific limitation of the current result and define an evaluation protocol for testing genuine generalization on unseen problems.

2 Related Works

Mathematical reasoning with language models has evolved from direct answer prediction toward chain-of-thought generation, tool-assisted reasoning, retrieval-augmented generation, verifier-guided search, and symbolic-neural hybrids. Benchmark suites such as MATH have highlighted the gap between linguistic fluency and reliable multi-step mathematical reasoning. At the same time, symbolic systems such as SymPy provide exact algebraic manipulation and arithmetic verification that complement neural models.

Retrieval-augmented generation introduces external knowledge into the reasoning context, but retrieval can also be harmful when context is irrelevant or only superficially similar to the current problem. This motivates retrieval gating and family-aware retrieval rather than unconditional context injection.

A related line of work uses external tools, program execution, or formal systems to reduce arithmetic and logical errors. Our work follows this direction but places particular emphasis on *routing*: exact tools are not merely optional helpers called by the model; they are first-class solvers selected before expensive neural inference whenever the mathematical family is recognized.

Another important area is verifier-based reasoning. In many existing pipelines, one language model generates a solution and another language model evaluates it. Although this can improve robustness, agreement between two neural models is not equivalent to mathematical proof. Our system therefore distinguishes neural critique from exact verification. Deterministic family engines produce explicit certificates, and candidate answers that lack decisive evidence remain unverified.

Finally, adaptive computation has become increasingly important as reasoning models grow more expensive. Rather than assigning the same reasoning budget to every question, our framework allocates computation according to problem structure and verification status. Easy exact problems can terminate with zero model calls, while unsupported hard problems can escalate to retrieval, deep reasoning, and critic review.

3 Methodology

3.1 Overall Framework

The proposed framework decomposes mathematical reasoning into six stages:

1. mathematical input recovery and fidelity checking;
2. semantic analysis and target extraction;
3. fine-grained problem-family routing;
4. deterministic exact solving when supported;
5. adaptive neural reasoning when exact solving is unavailable;
6. candidate verification and final answer certification.

The high-level flow is shown conceptually below:

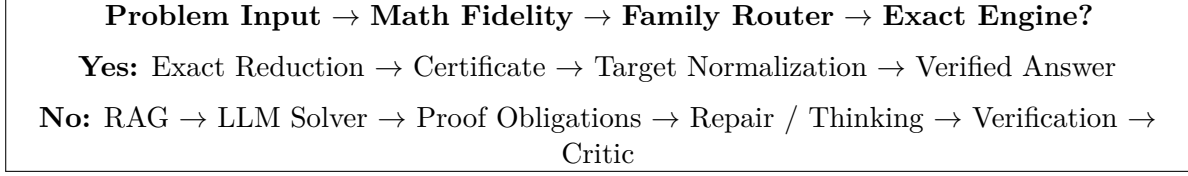


Figure 1: Conceptual architecture of the adaptive hybrid mathematical reasoning pipeline.

The framework is intentionally fail-closed. A candidate is not marked as verified merely because it satisfies the answer format or because a critic agrees with it.

3.2 System Architecture

The proposed system is designed as an adaptive hybrid reasoning pipeline rather than a conventional end-to-end language-model solver. Its architecture separates mathematical understanding, structural routing, deterministic computation, neural reasoning, and verification into independent stages. This design aims to improve both correctness and computational efficiency by ensuring that expensive language-model inference is only used when exact symbolic or algorithmic reasoning is unavailable.

The complete architecture consists of six major modules:

1. **Mathematical Input Fidelity Layer,**
2. **Problem Analysis and Target Parsing,**
3. **Fine-Grained Family Router,**
4. **Deterministic Exact Solver Layer,**
5. **Adaptive Neural Reasoning Layer,**
6. **Verification and Certification Layer.**

A conceptual overview of the architecture is shown in Fig. 2.

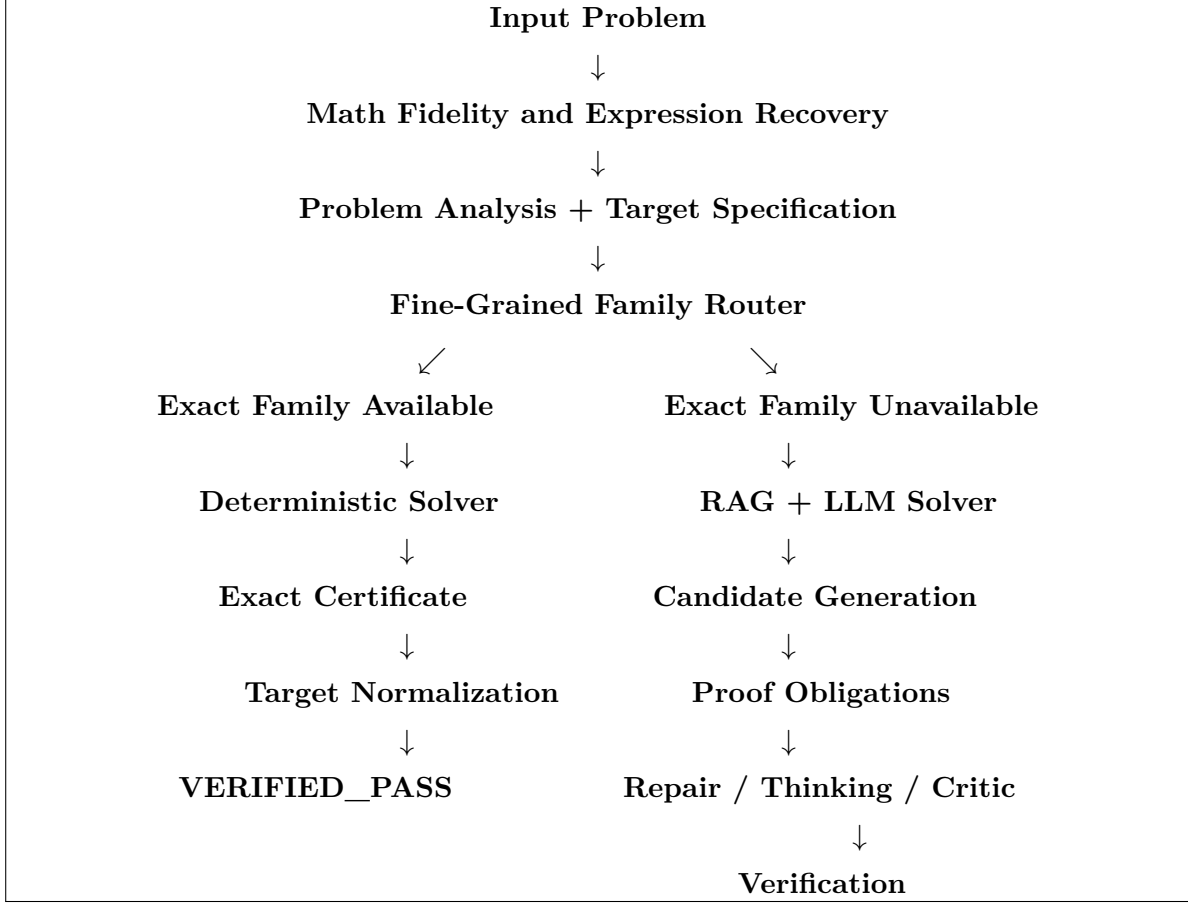


Figure 2: Overall architecture of the adaptive hybrid mathematical reasoning system.

3.3 Model Roles

The neural component contains three roles.

Lightweight proposer. DeepSeek-R1-Distill-Qwen-1.5B is used as an inexpensive strategy proposer when lightweight decomposition is useful.

Main solver and reviser. Qwen3-4B-Instruct-2507 acts as the primary structured solver, semantic compiler, proof repair model, and answer formatter.

Thinking critic. Qwen3-4B-Thinking-2507 is reserved for expensive deep reasoning, contradiction analysis, and critic review. It is not invoked when a deterministic solver already provides a complete certificate.

This role separation is designed to reduce unnecessary use of the highest-cost reasoning model.

3.4 Mathematical Input Fidelity

PDF extraction is a major source of failure in mathematical reasoning. A visually correct expression may be converted into a malformed linear text sequence. Typical examples include:

- superscripts detached from bases;
- fractions split across multiple lines;
- summation limits reordered;
- square roots or piecewise definitions corrupted;
- control characters inserted into formulas;
- geometry ratios flattened into ambiguous text.

The proposed math-fidelity stage therefore compares semantic extraction with layout cues and applies targeted recovery rules for recognized structures. The normalized mathematical problem is then validated before downstream reasoning.

An important design rule is:

If a critical mathematical expression cannot be recovered confidently, the system should stop or fall back to a safer recovery path rather than reason from corrupted input.

3.5 Fine-Grained Family Router

Broad labels such as `number_theory` or `geometry` were found to be insufficient for selecting an appropriate solver. The final system uses method-level families such as:

- `relational_algebra`;
- `extremal_partition`;
- `euclidean_geometry`;
- `functional_equation`;
- `combinatorial_process`;
- `floor_sum_valuation`;
- `geometry_sequence_asymptotic`;
- `adaptive_digit_sum_dynamics`;
- `finite_sequence_correlation`;
- `divisor_minimization_asymptotic`.

Each family is associated with a specialized hint, proof obligations, difficulty floor, and optionally an exact engine.

3.6 Target Specification

The target parser extracts the mathematical quantity requested by the problem and any modulus restriction.

For a modular target, a valid candidate must satisfy

$$0 \leq a < m, \tag{3}$$

where m is the requested modulus. Importantly, this check validates only answer format and range; it does not prove mathematical correctness.

3.7 Deterministic Family Engines

A deterministic family engine is invoked before neural reasoning when the structure is sufficiently specific. Every engine is fail-closed: it may decline a problem if required assumptions cannot be established.

3.7.1 Extremal Rectangle Partition

The engine for the rectangular partition problem separates the proof into an upper bound and a constructive lower bound. For each candidate semiperimeter it computes the minimum possible integer rectangle area, derives an area-based upper bound, and searches for a realizable strip construction. A result is certified only when the constructed count equals the proven upper bound and the tiling is mechanically validated for integer dimensions, containment, non-overlap, total area, and pairwise-distinct perimeters.

3.7.2 Exact Euclidean Geometry

For the geometry problem involving two intersecting circles and a minimal integer triangle, the configuration is reduced to an algebraic relation among side lengths. With

$$a = BC, \quad b = CA, \quad c = AB, \quad (4)$$

the exact condition becomes

$$a^2b = (b + c)^2(b - c). \quad (5)$$

The engine then searches integer triples subject to triangle inequalities, the acute-triangle condition, $c < b$, and the exact relation, stopping at the minimum perimeter.

3.7.3 Shifted Multiplicative Functional Equation

For

$$f(m) + f(n) = f(m + n + mn), \quad (6)$$

the substitution

$$g(t) = f(t - 1) \quad (7)$$

transforms the equation into

$$g(ab) = g(a) + g(b). \quad (8)$$

Thus g is completely additive on the positive integers and is determined by its values on primes. The exact engine converts bounded constraints into inequalities on prime weights and enumerates the feasible target values.

3.7.4 Binary Weighted Tournament

The tournament engine exploits the fact that distinct powers of two encode round histories uniquely. It compresses the tournament into score-history states, derives the counting formula for possible final orderings, and computes the relevant 2-adic and 5-adic valuations exactly.

3.7.5 Floor-Sum Valuation

The floor-sum engine uses Hermite's identity,

$$\sum_{i=1}^n \left\lfloor x + \frac{n-i}{n} \right\rfloor = \lfloor nx \rfloor. \quad (9)$$

For the reference structure,

$$f(n) = \sum_{i=1}^n \sum_{j=1}^n j^{1024} \left\lfloor \frac{1}{j} + \frac{n-i}{n} \right\rfloor \quad (10)$$

$$= \sum_{j=1}^n j^{1024} \left\lfloor \frac{n}{j} \right\rfloor \quad (11)$$

$$= \sum_{m=1}^n \sigma_{1024}(m). \quad (12)$$

Therefore,

$$f(N) - f(N-1) = \sigma_{1024}(N), \quad (13)$$

after which multiplicativity, exact valuation identities, and modular exponentiation complete the computation.

3.7.6 Geometry–Fibonacci Asymptotic

The geometry-sequence engine first reduces the geometric target ratio to an exact expression in Fibonacci terms. The resulting sequence is simplified symbolically and its asymptotic behavior is analyzed before the final modular target is evaluated. The engine explicitly avoids guessing a golden-ratio expression merely because Fibonacci numbers are present.

3.7.7 Adaptive Digit-Sum Dynamics

For the variable-base digit-sum process, the exact outgoing neighbors of an integer n are characterized as

$$\{1, 2, \dots, \lceil n/2 \rceil\}. \quad (14)$$

If $H(n)$ denotes the longest path length from n to 1, then

$$H(1) = 0, \quad (15)$$

and

$$H(n) = 1 + \max_{1 \leq r \leq \lceil n/2 \rceil} H(r). \quad (16)$$

The exact closed form is

$$H(n) = \lceil \log_2 n \rceil. \quad (17)$$

The implementation uses integer arithmetic rather than floating-point logarithms.

3.7.8 Finite Sequence Correlation

For finite-support integer sequences, define

$$P_\alpha(x) = \sum_t \alpha(t)x^t, \quad Q_\beta(x) = \sum_t \beta(t)x^{-t}. \quad (18)$$

The shifted inner-product condition becomes

$$P_\alpha(x)Q_\beta(x) = x^k + x^\ell. \quad (19)$$

The problem is therefore reduced to classifying polynomial divisors of

$$x^a(x^b + 1). \quad (20)$$

Cyclotomic factorization and degree-bounded enumeration then give an exact finite count.

3.7.9 Divisor-Minimization Asymptotic

For the Norwegian-number problem, the engine first classifies the minimizing divisor triples defining $f(n)$. For huge inputs of the form

$$M + c, \quad M = 3^{2025!}, \quad (21)$$

small-factor information is transferred through modular arithmetic instead of constructing M . Exact rational coefficients are obtained for $f(M + c)$, the floor term is bounded rigorously, and the final rational sum is computed with exact fraction arithmetic.

3.8 Semantic Compiler

Not all problems require a dedicated family engine. For compact algebraic word problems, a semantic compiler extracts entities, variables, relations, transfer events, domains, and the target quantity.

The compiler then validates the representation and solves the resulting system symbolically. A semantic result is accepted only when the derived certificate is connected directly to the requested target.

3.9 Retrieval-Augmented Reasoning

The framework distinguishes *knowledge retrieval* from *experience memory*.

Knowledge retrieval stores theorem summaries, identities, strategy patterns, and analogous problems. Experience memory stores previous trajectories, successful or failed strategies, and verification outcomes.

The current lightweight retrieval implementation uses word- and character-level TF-IDF features. Retrieved context is gated using statuses such as ACCEPT, WEAK, CONFLICT, and NO_CONTEXT. Retrieval is not used when a deterministic exact engine has already solved the problem.

3.10 Adaptive Compute Policy

The compute policy is hierarchical:

- exact-supported problem $\rightarrow$ deterministic engine;
- easy unsupported problem $\rightarrow$ short instruction-tuned reasoning;

- hard unsupported problem $\rightarrow$ retrieval, solver, proof repair, and optional thinking critic.

This policy is designed to reduce both tokens and GPU latency.

3.11 Proof Obligations and Verification

Each family defines explicit proof obligations. For example, the finite-correlation family requires:

- correlation-to-polynomial transformation;
- cyclotomic classification;
- degree-bounded enumeration;
- target count.

A candidate may be `CANDIDATE_ONLY`, `REFUTED`, `UNRESOLVED`, or `VERIFIED_PASS`. Early answer commitment is allowed to preserve a candidate under truncation, but an early commitment is never interpreted as verification.

If a proof checker refutes a candidate, the candidate is permanently excluded. Any suggested correction is treated as a new hypothesis and must be verified independently.

4 Experiments

4.1 Evaluation Dataset

The experimental benchmark consists of ten AIMO3 reference problems spanning algebra, combinatorics, geometry, functional equations, valuation, sequence asymptotics, discrete dynamics, polynomial correlation, and divisor minimization.

The benchmark answers are used only for evaluation and regression checks. They are not exposed to the runtime solver.

4.2 Execution Environment

The target execution environment is Kaggle with two NVIDIA Tesla T4 GPUs. Neural inference is performed offline using local checkpoints. Exact mathematical components execute primarily on the CPU.

The model allocation is:

- GPU 0: DeepSeek-R1-Distill-Qwen-1.5B and Qwen3-4B-Instruct-2507;
- GPU 1: Qwen3-4B-Thinking-2507;
- CPU: PDF recovery, retrieval, SymPy, exact arithmetic, family engines, and verification.

4.3 Evaluation Metrics

We report:

- candidate correctness;
- verified correctness;

- verification type;
- number of model calls;
- total tokens;
- measured latency;
- verification false positives.

For a set of N problems, verified accuracy is

$$\text{VerifiedAccuracy} = \frac{\#\{\text{correct verified answers}\}}{N}. \quad (22)$$

We also track exact-engine coverage,

$$\text{ExactCoverage} = \frac{\#\{\text{problems solved by exact family engines}\}}{N}. \quad (23)$$

5 Results and Discussion

5.1 Overall Performance

Table 1 summarizes the final batch.

Table 1: Final AIMO3 reference-set results.

Problem	Answer	Verified	Verification	Calls	Tokens	Latency (s)
P1	50	Yes	Semantic Exact	1	744	21.918
P2	520	Yes	Family Exact	0	0	0.107
P3	336	Yes	Family Exact	0	0	0.001
P4	580	Yes	Family Exact	0	0	0.272
P5	21818	Yes	Family Exact	0	0	0.002
P6	32951	Yes	Family Exact	0	0	0.008
P7	57447	Yes	Family Exact	0	0	0.045
P8	32193	Yes	Family Exact	0	0	0.006
P9	160	Yes	Family Exact	0	0	0.055
P10	8687	Yes	Family Exact	0	0	0.003

All ten problems reached `VERIFIED_PASS`. Problem 1 used semantic compilation and exact symbolic reasoning, requiring one model call and 744 tokens. Problems 2–10 were solved by deterministic family engines with zero model calls and zero neural tokens.

Across the complete batch, the recorded totals are:

- verified accuracy: 10/10;
- total model calls: 1;
- total neural tokens: 744;
- total recorded latency: approximately 22.419 seconds;
- verification false positives: 0.

The result demonstrates the potential efficiency of adaptive routing when exact solvers are available.

5.2 Problem-Specific Routing

The final routing behavior is summarized in Table 2.

Table 2: Primary reasoning route for each reference problem.

Problem	Main reasoning route
P1	Semantic compiler + exact symbolic solve
P2	Extremal rectangle partition
P3	Radical-axis / angle-bisector geometry engine
P4	Shifted multiplicative functional-equation engine
P5	Binary weighted tournament engine
P6	Floor-sum valuation engine
P7	Geometry-Fibonacci asymptotic engine
P8	Adaptive digit-sum dynamics engine
P9	Finite sequence correlation / polynomial engine
P10	Divisor-minimization asymptotic engine

This routing diversity is important. The benchmark is not solved by a single mathematical template; instead, the architecture maps different structures to different exact mechanisms.

5.3 Why the Hybrid Architecture Improved Reliability

The development process revealed that several early failures were not caused by insufficient model size alone.

Geometry hallucination. In an earlier version, the geometry solver guessed familiar integer triangles without deriving the exact geometric condition. The deterministic geometry engine removed this failure mode by deriving a side-length equation and searching only certified integer triples.

Floor-sum reasoning error. The model previously made an incorrect inequality claim inside a floor expression. Replacing free-form reasoning with Hermite’s identity and divisor-sum reduction eliminated the ambiguity.

Sequence-dynamics truncation. For the digit-sum process, the language model repeatedly produced long heuristic arguments and truncated before a valid candidate. The exact transition characterization reduced the problem to a simple recurrence and integer logarithm.

Malformed correlation operator. The finite-sequence problem initially suffered from PDF extraction that separated the summation sign from its operands. The math-recovery layer restored the shifted inner product before the polynomial engine was invoked.

Approximate reasoning inside an exact floor. For the divisor-minimization problem, an earlier candidate relied on an unsupported approximation of $f(M + c)$. The final solver instead classifies the exact minimizing structure and proves the floor stabilization.

These examples suggest that reliability can be improved significantly by correcting the *reasoning substrate*: input representation, solver choice, and verification.

5.4 Compute Efficiency

The strongest efficiency observation in the final batch is that nine of ten problems require no neural inference after routing. The first problem requires one compact semantic-compilation call, while all remaining problems terminate through exact engines.

This does not imply that every unseen Olympiad problem will admit a pre-existing exact family. Rather, it demonstrates the value of a hierarchical policy:

$$\text{exact computation first} \rightarrow \text{neural reasoning only when necessary.} \quad (24)$$

For a larger unseen benchmark, exact-engine coverage will become a key variable determining total compute.

5.5 Verification Behavior

No incorrect answer in the final batch was marked as verified. This reflects an important distinction between answer generation and answer certification.

The system does not interpret the following as sufficient evidence:

- a high model confidence score;
- a syntactically valid integer answer;
- agreement between two language models;
- a candidate suggested by a proof-repair step.

Instead, a deterministic exact path must satisfy its family-specific certificate. Neural-only paths remain subject to proof obligations and critic review.

5.6 Interpretation of the 10/10 Result

The reference benchmark was used iteratively during system development. Consequently, the final 10/10 verified result is an *engineering validation result*, not an unbiased generalization result.

This distinction is central to the scientific interpretation of the project. Several deterministic solvers were introduced after analyzing specific failure modes on these reference problems. Therefore, the current result demonstrates that the architecture can represent and solve a diverse collection of mathematical structures after refinement, but it does not yet establish how well the same frozen system transfers to unseen tasks.

6 Ablation and Evaluation Plan

A complete research evaluation should compare the proposed adaptive system against controlled baselines.

6.1 Recommended Baselines

1. Qwen3-4B-Instruct with fixed prompting and fixed token budget;
2. Qwen3-4B with retrieval but no exact engines;
3. Qwen3-4B with tools but no adaptive router;
4. Qwen3-4B adaptive hybrid system;
5. a larger fixed-reasoning baseline, e.g., a 7B model.

6.2 Recommended Ablations

The following ablations isolate the contribution of individual components:

- no math-recovery layer;
- no retrieval;
- no exact engines;
- no critic;
- no adaptive compute;
- solver-only;
- thinking-only;
- coarse router instead of fine-grained family routing.

6.3 Recommended Metrics

In addition to accuracy, future experiments should report:

- verified accuracy;
- false-verification rate;
- average model calls per problem;
- average input/output tokens;
- GPU inference time;
- wall-clock latency;
- exact-engine coverage;
- retrieval-usefulness and retrieval-harm rates.

7 Limitations

The current system has several limitations.

First, the AIMO3 reference set was used during iterative engineering. The exact engines therefore encode mathematical families that were directly motivated by observed failures. A frozen unseen benchmark is required before making claims about generalization.

Second, the current lightweight retrieval layer uses a relatively small TF-IDF knowledge store. A stronger retrieval system could combine dense embeddings, BM25, symbol matching, domain agreement, and contradiction-aware gating.

Third, deterministic engines currently rely on recognizable mathematical structure. Problems that fall outside supported families still require neural reasoning. The long-term goal is therefore not to create one engine per benchmark problem, but to develop increasingly general symbolic compilers and reusable solver abstractions.

Fourth, neural critic output is not formal proof. Although the system uses deterministic certificates where possible, unsupported neural paths would benefit from deeper integration with theorem provers, SMT solvers, or proof-producing symbolic systems.

Finally, the present paper reports a development benchmark rather than a statistically large evaluation. More problems and multiple model baselines are required for stronger empirical conclusions.

8 Conclusion

8.1 Concluding Remarks

This work presented an adaptive hybrid framework for compute-efficient Olympiad mathematical reasoning. The central idea is to avoid treating every problem as an unrestricted language-model generation task. Instead, the system recovers mathematical input, identifies problem structure, routes supported families to exact solvers, allocates neural compute adaptively, and separates candidate generation from verification.

On the ten AIMO3 reference problems used during development, the final system verifies all ten answers. Nine problems are solved entirely by deterministic family engines, while one uses a semantic compiler with a single neural call. The final batch contains no verification false positives.

The most important conclusion is not that exact engines can solve a particular benchmark, but that *reasoning quality depends strongly on the architecture surrounding the language model*. Input fidelity, routing, tool selection, symbolic computation, and verification can be as important as raw model size.

8.2 Future Research

The next phase should freeze the v2.11 architecture and evaluate it on unseen Olympiad problems. The key experimental question is whether the same system can maintain or improve accuracy while reducing tokens, model calls, and GPU time relative to fixed-compute baselines.

Future work will focus on:

- generalized floor-sum and valuation compilers;
- automatic recurrence discovery;
- reusable polynomial-divisibility solvers;

- geometry-to-coordinate compilation;
- learned family routing;
- learned retrieval gating;
- structured experience memory;
- formal verification with theorem provers.

Ultimately, this project investigates a broader hypothesis: small and medium language models may achieve substantially stronger mathematical reasoning when embedded inside a system that combines language understanding, adaptive search, exact computation, and independent verification.

References

- [1] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, Measuring Mathematical Problem Solving With the MATH Dataset, *NeurIPS*, 2021.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, *NeurIPS*, 2020.
- [3] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, Š. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, SymPy: symbolic computing in Python, *PeerJ Computer Science*, 3:e103, 2017.
- [4] DeepSeek-AI, DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, arXiv preprint, 2025.